Introduction to Programming Languages

Friday, April 1, 2022 4:57 PM

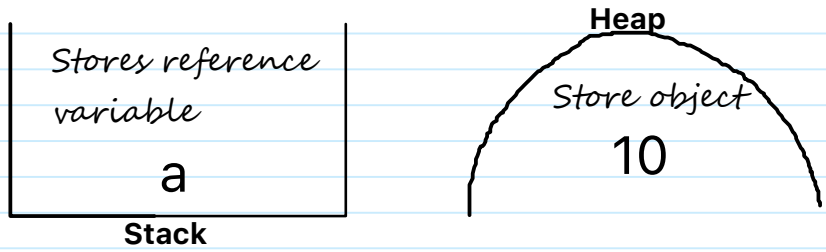
Types of Programming Languages

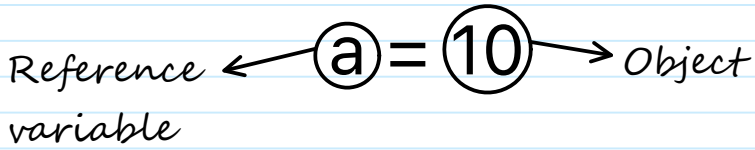
- Procedural
- Functional
- Object Oriented

Static Languages = Perform type checking at compile time
Dynamic Languages = Perform type checking at runtime

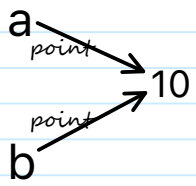
Memory Management

There are 2 types of memory **Stack** and **Heap**



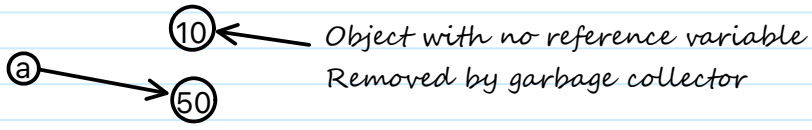


a = 10
b = a



- More than one reference variable can point towords one object.
  If any of the reference variable changes the object then it is changed
- For all reference variable that points towords same object.

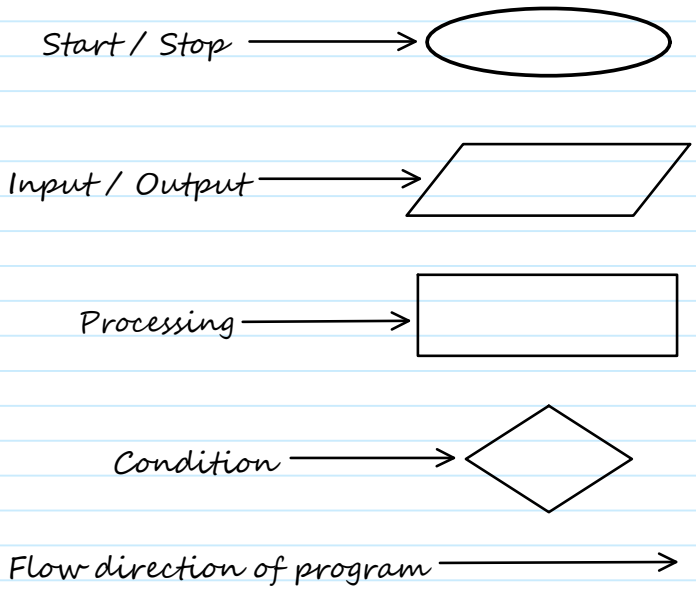
Now intially,
a = 10
a = 50



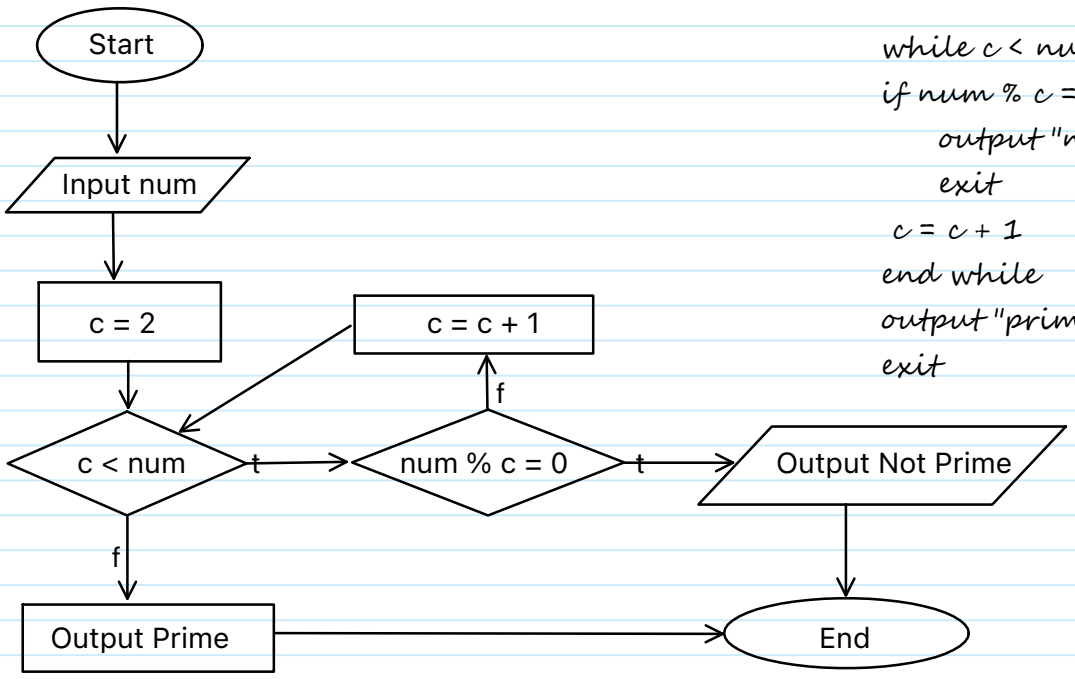
Flow of Program

Friday, April 1, 2022 7:28 PM

FlowCharts



Input a number and print whether it is prime or not.




```
while c < num;
if num % c = 0;
    output "not prime"
    exit
 c = c + 1
end while
output "prime"
exit
```

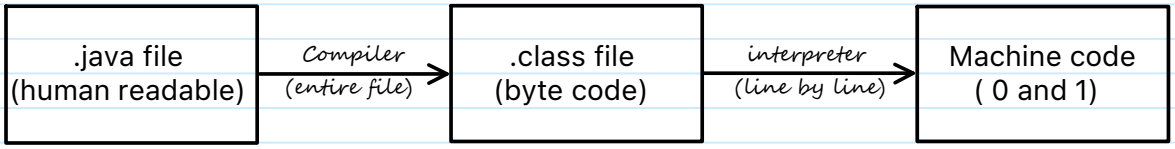
So, while c*c <= num;

1 x 36 = 36
2 x 18 = 36
3 x 12 = 36
4 x 9 = 36
6 x 6 = 36
9 x 4 = 36
~~12 x 3 = 36~~
~~18 x 2 = 36~~
~~36 x 1 = 36~~

Introduction to java

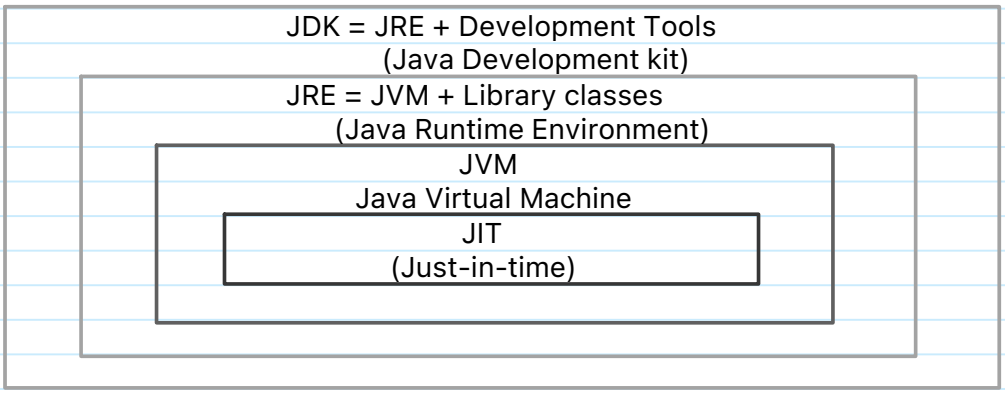
Saturday, April 2, 2022 9:48 AM

How java code executes



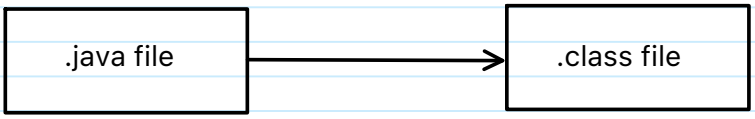
Java Compiler:- This byte code not directly run on system, We need JVM to run this, Reason why java is platform independent

Java Interpreter:- Converts byte code to machine code i.e. 0's and 1's, It translate the byte code line by line to machine code

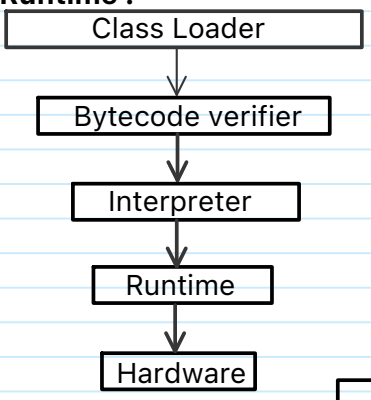


JDK :- Provide Environment to develop and run the java program
JRE :- It is an installation package that provides environment to only run the program

Compile Time :-



Runtime :-



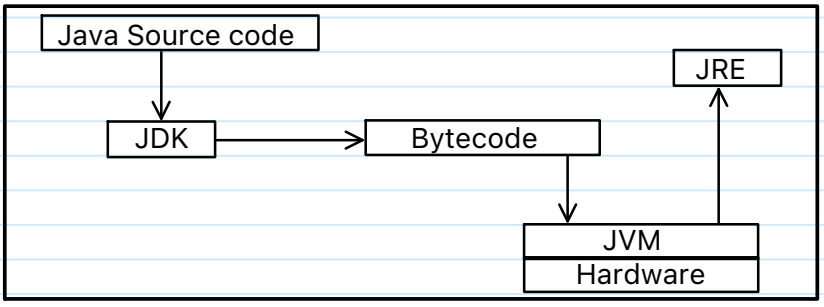
Class Loader (How JVM Works) :-

Loading → Read .class file and generate binary data.
Linking → JVM verifies the .class file
→ Allocates memory for class variables and default values
Initialization→ JVM contains the stack and heap memory locations.

JVM Execution :-

Interpreter → Line by line execution, method is called many times
JIT → Those methods that are repeated, JIT provides direct machine code so that interpretation is not required. →Makes execution Faster.
→ Garbage collector

Working of Java Architecture :-



First Java Program

Tuesday, April 5, 2022 1:45 PM

Converting .java to .class

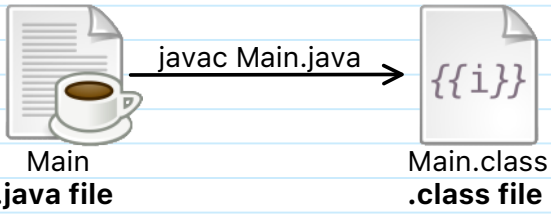
- Using javac compiler we can convert .java file to .class
- Command to convert .java to .class

Javac and .java file name

- Let the name of .java file is Main,

Javac Main.java

- . class file contains Bytecode



File name: Demo.java
Class name: Demo

- Run the program,

java Main

Hello world program

```
public class Main{
        public static void main(String [] args){
                System.out.println("Hello World");
        }
}
```

1. **public (in first line)** :- public is an access modifier which allows to access the class from anywhere.
2. **class** :- It is a name group of properties and functions
3. **Main** :- It is just the name of class as same as the name of file.
4. **public (in second line)** :- It is used to allow the program to use main function from anywhere.
5. **static** :- It is a keyword which helps the main method to run without using objects.
6. **void** :- It is a keyword used when we do not want to return anything from a method/function
7. **main** :- It is the name of method.
8. **String [] args** :- It is a command line argument of string type array.
9. **System** :- It is a final class defined in java.lang package.
10. **out** :- It is a variable of PrintStream type which is public and static member field of the System class.
11. **println** :- It is a method of PrintStream class, It prints the argumants passed to it and adds a new line. **print** can aalso be used here but it prints only arguments passed to it. It do not adds a new line.
12 **Package** :- → It is just a folder in which java files lies.
→ It is used to provide some rules and stuff to our programs.

Data types	Description	Example
int	int is used to store numeric digits	int i = 26;
char	char is used to store character	char c = 'A';
float	float is used to store floating point numbers	float f = 98.67F;
double (by default)	double is used to store larger decimal numbers	double d = 45676.58975;
long	long is used to store numeric digits which is not able to stored in int	long l = 15876954832558315L;
boolean	It only stores store t values i.e., true or false.	boolean b = false;

Primitives data types are those data types which is not breakable, Ex. char ,int etc.

- **Literals** :- It is a synthetic representation of boolean, character, string, and numeric data. Ex:- int a = 10; Here 10 is called literal.
- **Identifiers**:- name of variable, methods, class, packages, etc. are known as identifiers. Ex:- int a = 10; Here a is Identifier.

Comment:- `//, /* */` and `int a = 23_00_00;` o/t `230000` underscore will be ignored

Inputs in Java

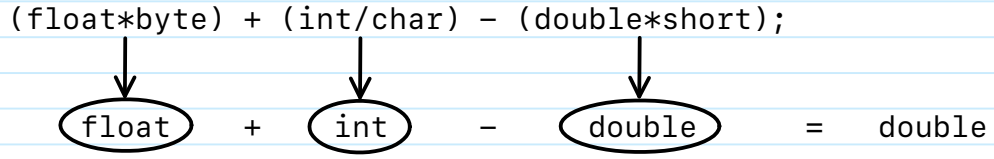
```
import java.util.Scanner;
public class Main{
        public static void main(String [] args){
                Scanner input = new Scanner(System.in);
                int rollno = input.nextInt();
                String s1 = input.next();
                String s2 = input.nextLine();
                System.out.println(rollno + " " + s1 + " " + s2);
        }
}
```

1. **Scanner** :- It is a class required to take input, it is present in java.util package.
2. **input** :- It is an object that we are creating to take input.
3. **new** :- It is a keyword used to create an object in java.
4. **System.in** :- System is a class and in is a variable that denotes we are taking input from standard input stream (i.e. Keyboard).

Type conversion :- When one type of data is assigned to another type of variable an automatic type conversion will take place under some condition

1. Two types should be compatible.
2. Destination type should be greater then the source type.

byte -> short -> int -> long -> float -> double



Type Casting :- When we convert one type of data to another type is known as type casting
Ex:- int num = (int) (67.564f)

Prime number program

```
public static void main(String[] args){
     Scanner in = new Scanner(System.in);
     System.out.println("Please enter a number);
     int n = in.nextInt();
     if(n<=1){
          System.out.println("Nither prime nor composite");
          return;
     }
     int c = 2;
     if(n==4){
          System.out.println("Not Prime");
     }
     else{
          while(c*c<n){
               if(n%c==0){
                   System.out.println("Not Prime");
                   return;
               }
               c=c+1;
          }
          if(c*c>n){
               System.out.println("Prime");
          }
      }
}
```

Output :-

```
Please enter a number
17
Prime
```

Celsius to Fahrenheit program.

```
public static void main (String[] args){
    Scanner in = new Scanner(System.in);
    float tempC = in.nextFloat();
    float tempF = (tempC*9/5) + 32;
    System.out.println("Temperature in fahrenheit: " + tempF);
}
```

Output :-

```
45
Temperature in fahrenheit: 113.0
```

Conditional and loops

Wednesday, April 6, 2022 9:37 AM

Condition :- It provide check for the statement.

If-else statement → Used to check the condition, it checks the Boolean condition True of False.

Example :-

```
public static void main(String[] args){
    int salary = 25400;
    if (salary<= 10000) {
        salary +=1000;
    }
    else if (salary <= 20000) {
        salary += 2000;
    }
    else {
        salary += 3000;
    }
    System.out.println(salary);
}
```

Output :- 28400

Loop :- Loops are used to iterate a part of program several times.

for loop → It is generally used when we know how many times loop will iterate.

Example:- print numbers from 1 to 5

```
public static void main(String[] args){
   for (int num=1;num<=5;num++){
          System.out.print(num + " ");
   }
}
```

Output :- 1 2 3 4 5

while Loop → It is used when we don't know how many time the loop will iterate.

Example :-

```
public static void main(String[] args) {
    int num = 1;
    while (num <=5){
        System.out.print(num + " ");
        num ++;
    }
}
```

Output :- 1 2 3 4 5

do while loop → It is used when we want to execute our statement at least one time.
→ It is called exit control loop because it checks the condition after execution of statement.

Example :-

```
public static void main(String[] args) {
   int n = 1;
   do{
       System.out.println(n + " ");
       n++;
   } while(n<=5);
}
```

Output :- 1 2 3 4 5

Alphabet case check

```
import java.util.Scanner;
public class AlphabetCaseCheck {
    public static void main(String[] args) {
        Scanner in = new Scanner (System.in);
        char ch = in.next().trim().charAt(0);
        if (ch > 'a' && ch <= 'z'){
            System.out.println("Lowercase");
        }
        else {
            System.out.println("Uppercase");
        }
    }
}
```

Input :- a
Output :- Lowercase
Input :- Z
Output :- Uppercase

Fibonacci Numbers :- a series of numbers in which each number (Fibonacci number) is the sum of the two preceding numbers.

Ex :- 0,1,1,2,3,5,8,13...

Find the nth Fibonacci number.

```
import java.util.Scanner;
public class FibonacciNumbers{
    public static void main(String[] args) {
        Scanner in = new Scanner (System.in);
        int n = in.nextInt();
        int a = in.nextInt();
        int b = in.nextInt();
        int count = 2;

        while(count <= n){
            int temp = b;
            b = b+a;
            a = temp;
            count++;
        }
        System.out.println(b);
    }
}
```

0, 1, 1, 2, 3, 5, 8, 13,...

a	b
0	1
1	1
1	2
2	3
3	5

Input :- 0 1 7

Output :- 8.

Reverse a number

```
import java.util.Scanner;
public class ReverseANumber {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        int num = in.nextInt();
        int ans = 0;
        while(num > 0){
           int rem = num % 10;
           num /= 10;
           ans = ans * 10 + rem;
        }
        System.out.println(ans);
    }
}
```

n = 1 3 8 5 7 5 7 8 7 9

n % 10 = last digit

n = 1 3 8 9 / 10

Ans = 3

10) 1389 (138
- 1380
9

Input :- 458792

Output :- 297854

n = 23597

ans = 7 * 10 + 9 = 79

= 79 * 10 + 5 = 795

= 795 * 10 + 3 = 7,953

= 7953 * 10 + 2 = 79532

Ans : 79532

2 3 5 9 7

Switch Statements + Nested case in java

```
switch ( expression) {

        case one :

            //code block

            break;

        case two :

            //code block

            break;

        default :

            // code block

}
```

Terminate the sequence

→ default will execute when none of above does
→ if default is not at end put break after it

→ if break is not used then it will continue with other cases.
→ duplicate cases not allowed.

`x.equals("word")` → here equals only checks value not reference

`x == " word"` → here it checks reference

```
Scanner in = new Scanner(System.in);
int day = in.nextInt();

switch (day) {
        case 1:
        case 2:
        case 3:
        case 4:
        case 5:
            System.out.println("Weekday");
            break;
        case 6:
        case 7:
            System.out.println("Weekend");
            break;
    }

switch (day) {
        case 1, 2, 3, 4, 5 -> System.out.println("Weekday");
        case 6, 7 -> System.out.println("Weekend");
    }
```

```java
package com.parth;

import java.util.Scanner;

public class NestedSwitch {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        int empID = in.nextInt();
        String department = in.next();

        switch (empID) {
            case 1:
                System.out.println("Kunal Kushwaha");
                break;
            case 2:
                System.out.println("Parth Patel");
                break;
            case 3:
                System.out.println("Emp Number 3");
                switch (department) {
                    case "IT":
                        System.out.println("IT Department");
                        break;
                    case "Management":
                        System.out.println("Management Department");
                        break;
                    default:
                        System.out.println("No department entered");
                }
                break;
            default:
                System.out.println("Enter correct EmpID");
        }

        // better way to write

        switch (empID) {
            case 1 -> System.out.println("Kunal Kushwaha");
            case 2 -> System.out.println("Parth Patel");
            case 3 -> {
                System.out.println("Emp Number 3");
                switch (department) {
                    case "IT" -> System.out.println("IT Department");
                    case "Management" -> System.out.println("Management
                                         Department");
                    default -> System.out.println("No department entered");
                }
            }
            default -> System.out.println("Enter correct EmpID");
        }
    }
}
```

Functions/ Methods (in Java)

Thursday, April 7, 2022 10:29 AM

- A method is a block of code which only runs when it is called.
- To reuse code: define the code once, & use it many times.

This method myMethod() not have a return value.

Name of method

Syntax :

```
public class Main{
        static void myMethod(){
                // code
        }
}
```

Calling the function

Syntax :

```
public class Main{
        access_modifier return_type method(){
                // code
                return statement;
        }
}
```

name of function

function ends here
A return statement causes the program control to transfer back to the caller of a method.

Pass by value :

Object/value

```
main () {
        name = " a " ;
        greet ( name );
}
static void greet (naam) {

print(naam)
}
```

name → a
naam → a

Creating copy of value of name
i.e. passing value of the reference.

```
psvm(){

        name = "a";
        change(name);
        print(name);
}

change(naam){

        naam = "b";

}
```

Creating copy

name → a
naam → a

name → a
naam → b

Not changing original object, just creating new object.
Since it is created inside function it will not change original one.

Points to be noted:

1. primitive data type like int, short, char, byte etc. → just pass value
2. object & reference: → passing value of reference variable

eg-1:

```
psvm(){
        a = 10;                 a → 10     but not
        b = 20;                 b → 10     here
        swap(a,b);
}

swap(num1, num2){
        temp = num1;            temp → 10     at function
        num1 = num2;            num1 → 20     scope level
        num2 = temp;            num2 → 10     they are swapped
}
```

eg-2:

arr → [1, 2, 3, 4, 5]
nums ↗

num[0] = 99 [now, the value of 0th position in nums will change which also changes value of arr[0]]

arr → [99, 2, 3, 4, 5]
nums ↗

Here, passing value of reference variable

Scopes:

Function scope : Variables declared inside a method / function scope (means inside method) can't be accessed outside the method.

eg :-

```
psvm(){
        X
    }
all(){
        int x;
    }
```

can't be accessed outside

Block scope :

```
psvm(){
    int a = 10;
    int b = 20;
    {
        int a = 5;
        a = 100;
        int c = 20;
    }
    c = 10;
    int c = 15;
    a = 50;
}
```

variable initialized outside the block can be updated inside the box.

variables initialized inside the block cannot be updated outside the box but can be reinitialized outside the block.

variables like "a" here, is declared outside the block, updated inside the block and can also be updated outside the block.

Loop scope : Variables declared inside loop are having loop scope

Shadowing :

```
public class shadowing{
    static int x = 90;
    psvm(){
        System.out.println(x);
        x=50;      // here high-level scope is
        System.out.println(x);     shadowed by
    }                               low-level
}                                     scope
```

90

50

Variable Arguments(varargs) : Variable arguments is used to take a variable number of arguments. A method that takes a variable number of arguments is a varargs method.

Syntax :

```
static void fun(int…a){
        // method body
}
```

Here, would be array of type int[] parameters

Method / Function Overloading : function Overloading happens when two functions have same name.

eg :- (1)

```
fun(){
    // code
}
fun(){
    // code
}
```

✗ function overloding

(2)

```
fun(int a){
    // code
}
fun(int a, int b){
    // code
}
```

This is allowed having different arguments with same method name

→ At compile time, it decides with f'n to run.

Armstrong number :

suppose there is number → 153

$153 \rightarrow (1)^3+(5)^3+(3)^3 = 1 + 125 + 27$

$= 153$

Function

```
public class Sum {
    public static void main(String[] args) {
//          int ans = sum2();
//          System.out.println(ans);

        int ans = sum3(20, 30);
        System.out.println(ans);
    }

    // pass the value of numbers when you are calling the method in main()
    static int sum3(int a, int b) {
        int sum = a + b;
        return sum;
    }

   // return the value
    static int sum2() {
        Scanner in = new Scanner(System.in);
        System.out.print("Enter number 1: ");
        int num1 = in.nextInt();
        System.out.print("Enter number 2: ");
        int num2 = in.nextInt();
        int sum = num1 + num2;
        return sum;
//          System.out.println("This will never execute");
    }
}
```

Swap

```
public class Swap {
    public static void main(String[] args) {
        // create an array
        int[] arr = {1, 3, 2, 45, 6};
        change(arr);
        System.out.println(Arrays.toString(arr));

        String name = "Kunal Kushwaha";
        changeName(name);
        System.out.println(name);
    }
    static void change(int[] nums) {
        nums[0] = 99; // if you make a change to the object via this ref variable,
    }                    same object will be changed
    static void changeName(String naam) {
        naam = "Rahul Rana"; // creating a new object
    }
}
```

VarArgs

```
  public static void main(String[] args) {

          multiple(2, 3, "Kunal", "Rahul", "dvytsbhusc");

      }

  static void multiple(int a, int b, String ...v) {

      }
```

Overloading

```
  public class Overloading {
      public static void main(String[] args) {
  //        fun(67);
  //        fun("Kunal Kushwaha");
          int ans = sum(3, 4, 78);
          System.out.println(ans);
      }
      static int sum(int a, int b) {
          return a + b;
      }
      static int sum(int a, int b, int c) {
          return a + b + c;
      }
      static void fun(int a) {
          System.out.println("first one");
          System.out.println(a);
      }
      static void fun(String name) {
          System.out.println("Second one");
          System.out.println(name);
      }
  }
```

Print all the 3 digits Armstrong numbers

```
package com.kunal;

import java.util.Scanner;

public class Questions {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
//        int n = in.nextInt();
//        boolean ans = isPrime(n);
//        System.out.println(ans);

        for (int i = 100; i < 1000; i++) {
            if (isArmstrong(i)) {
                System.out.print(i + " ");
            }
        }
    }

    // print all the 3 digits armstrong numbers
    static boolean isArmstrong(int n) {
        int original = n;
        int sum = 0;

        while (n > 0) {
            int rem = n % 10;
            n = n / 10;
            sum = sum + rem*rem*rem;
        }

        return sum == original;
    }

    static boolean isPrime(int n) {
        if (n <= 1) {
            return false;
        }
        int c = 2;
        while (c * c <= n) {
            if (n % c == 0) {
                return false;
            }
            c++;
        }
        return c * c > n;
    }
}
```

Array & Arraylist

Saturday, April 9, 2022 10:12 AM

An array is a collection of similar data type values.

Syntax of an Array :

datatype[] variable_name = new datatype[size]

eg :-

```
int[] rollno = new int[5];
```

store 5 roll numbers

it will create an object in heap me. of object size 5

or

```
int[] rollno = {51, 82, 13, 15,16}
```

represent the type of data store in array.

All the type of data in array should be same!

```
int [] rollnos;   // declaration of array
```

rollno are gatting defined in stack

```
rollnos = new int [5]; // initialisation
```

actual memory allocation happens here

Here, object is being created in heap memory.

declaration of array — compile time — run time — initialisation

```
int [] arr = new int [5];
```

datatype — ref var — creating object in heap memory

This above concept is known as Dynamic memory allocation which means at runtime OR execution time memory is allocated.

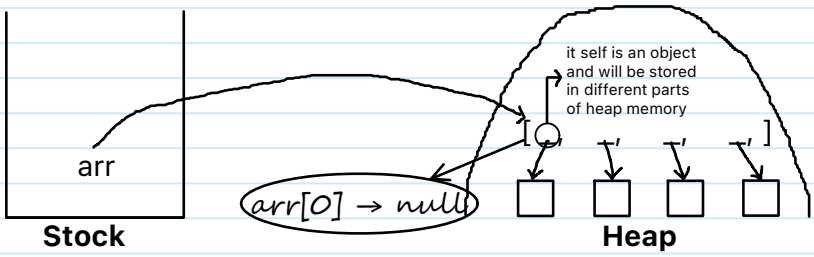
Internal Representation of Array :
internally in Java, memory allocation totally depends on JVM whether it be continuous or not!

- Objects are stored in heap memory.
- In JLS(Java Language specification) if is mentioned that heap objects are not continuous
- Dynamic memory allocation. Hence, array objects in Java may not be continous(depend on JVM)

Index of an array :-

index →

0	1	2	3	4	5
3	8	9	10	53	33



- Primitives (int, char etc) are stored in stored in stack.
- All other objects are stored in heap memory

Array.toString(array**)** → internally uses for loop and gives the output in proper format.

- *In an array, since we can change the objects, hence they are mutable.*
- *String are immutable.*

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

row column

⇒ `int [][] arr = new int[size][]`

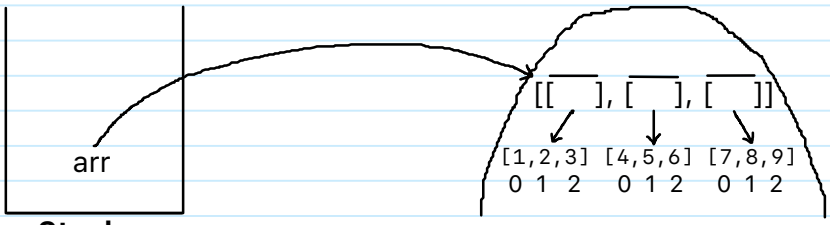
mandatory to give size of row

not mandatory to give size of column

or

```
int [][] arr = {
                {1, 2, 3},
                {4, 5, 6},
                {7, 8, 9}
               }
```

[arrays of arrays]



arr[0] = [1,2,3]
arr[0][2] = 3